---

first semester 2023/2024

Digital systems ENCS2340

ALU Verilog Project

Sadeen Jehad Faqeeh.

122217.

Sec.6

Dr. **Ahmed Shawahna.**

Introduction :

In this project, I designed a simple arithmetic logic unit (ALU) using Verilog Hardware Description Language (HDL). This design is capable of performing four basic arithmetic and logical operations: addition, subtraction, bitwise AND, and bitwise OR.

In this design there were several tasks. The tasks:

1. ALU Design: o Define inputs and outputs of the ALU. o Select four essential arithmetic and logical operations to implement (e.g., addition, subtraction, AND, OR). o Create a block diagram representing the ALU's structure.

2. Verilog Modules: o Develop a top-level ALU module with appropriate input/output ports. o Design separate modules for each of the four chosen operations, utilizing a combination of structural and behavioral modeling approaches. • Modules: Adder, Subtractor, AndGate, OrGate, and ALU (toplevel module). • The ALU design should be modular, comprising separate modules for each functionality. • Implement the modules using structure, dataflow, and behavioral modeling as below: 1. Utilize structural modeling for Adder, Subtractor, and logic gates. 2. Implement dataflow modeling for connecting the modules. 3. Utilize behavioral modeling for the top-level ALU module

. 3. Input/Output: o The ALU should take two 4-bit inputs (A and B). o Include a 3-bit control input (OpCode) to select the operation: • 3'b000: Addition • 3'b001: Subtraction • 3'b010: Bitwise AND • 3'b011: Bitwise OR o Output the result (Result) of the selected operation.

4. Testing and Simulation: o Test various combinations of input values and control codes to ensure the ALU operates as expected. Apply a variety of input test cases to cover all possible combinations. o Simulate the ALU Components/Modules using a Verilog simulator and analyze the results o Simulate the ALU using a Verilog simulator and analyze the results.

5. Documentation: o Provide clear and concise comments within the Verilog code for better understanding. o Prepare a well-formatted project report detailing the design process, code implementation, simulation results, and conclusions.

Modules:

The modiles that I used :

1. Bitwise AND => bitwiseAnd
2. Bitwise OR => bitwiseOr
3. Full adder for one bit=> fullAdderOB
4. Adder => add
5. Subtractor => sub
6. ALU (top-level module):
    - ALU using dataflow => dataflowALU
    - ALU using behavioral => behavioralALU

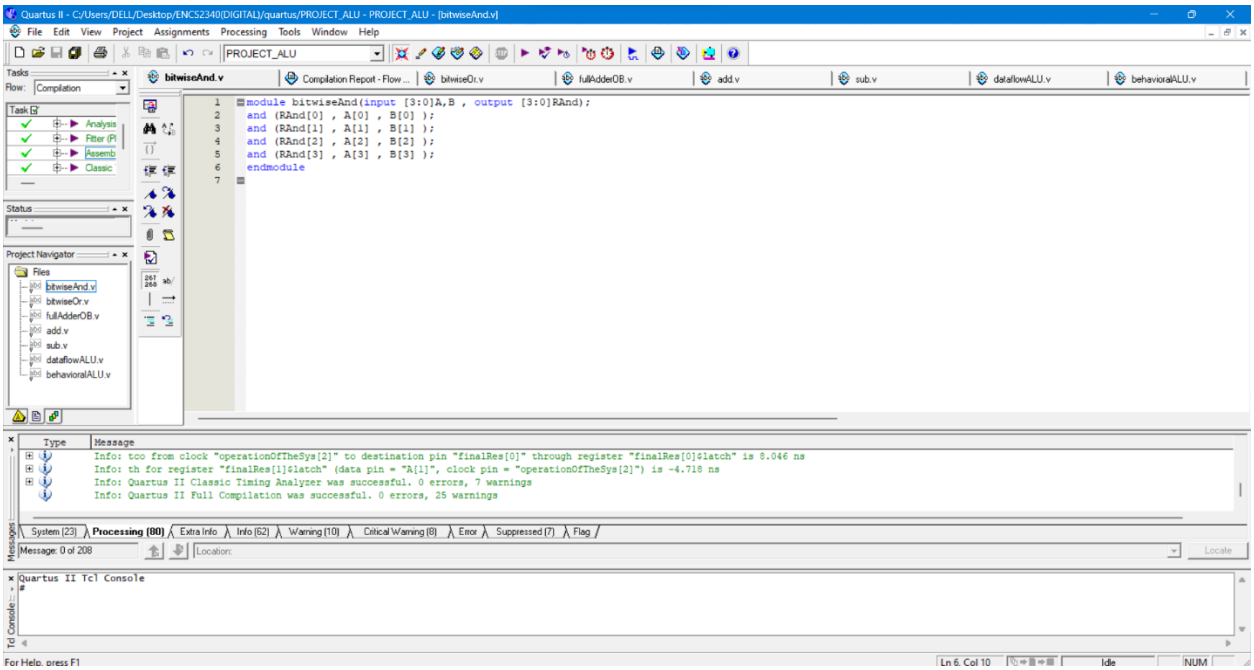
Components:

1. Bitwise AND (bitwiseAnd):

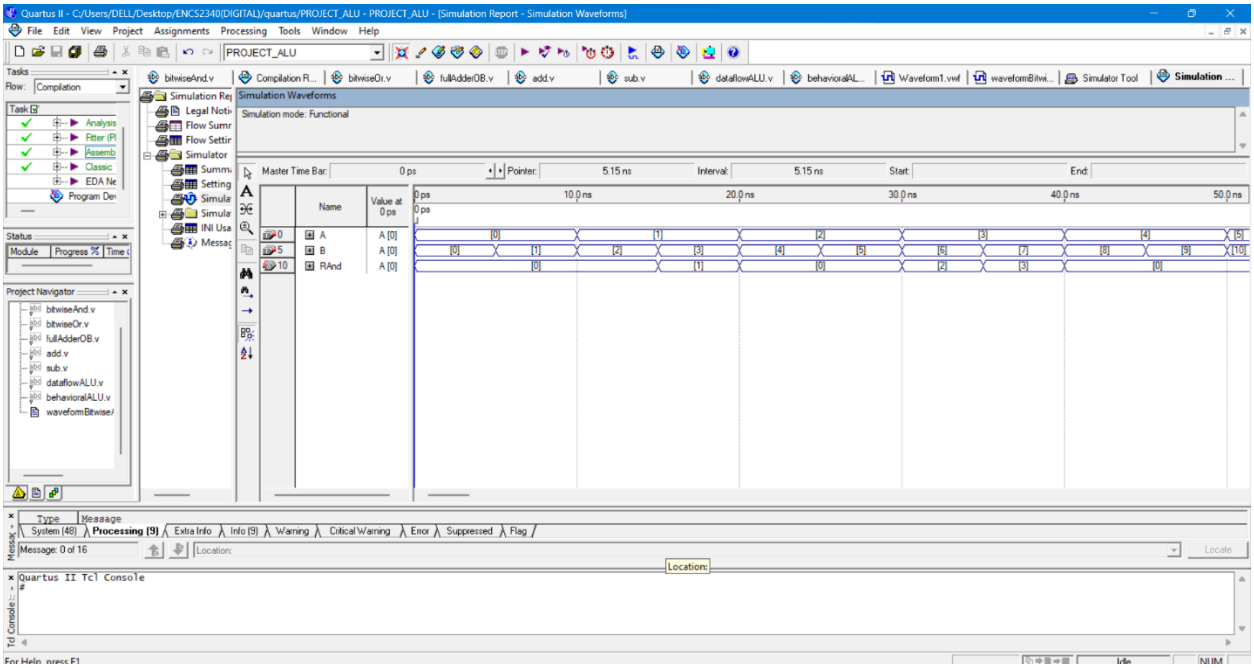
A **bitwise AND** is a binary operation that takes two equal-length binary representations and performs the logical AND operation on each pair of the corresponding bits. Thus, if both bits in the compared position are 1, the bit in the resulting binary representation is 1 (1 × 1 = 1); otherwise, the result is 0 (1 × 0 = 0 and 0 × 0 = 0).

2. The code :

```
module bitwiseAnd(input [3:0]A,B , output [3:0]RAnd);

and (RAnd[0] , A[0] , B[0] );

and (RAnd[1] , A[1] , B[1] );

and (RAnd[2] , A[2] , B[2] );

and (RAnd[3] , A[3] , B[3] );

endmodule
```

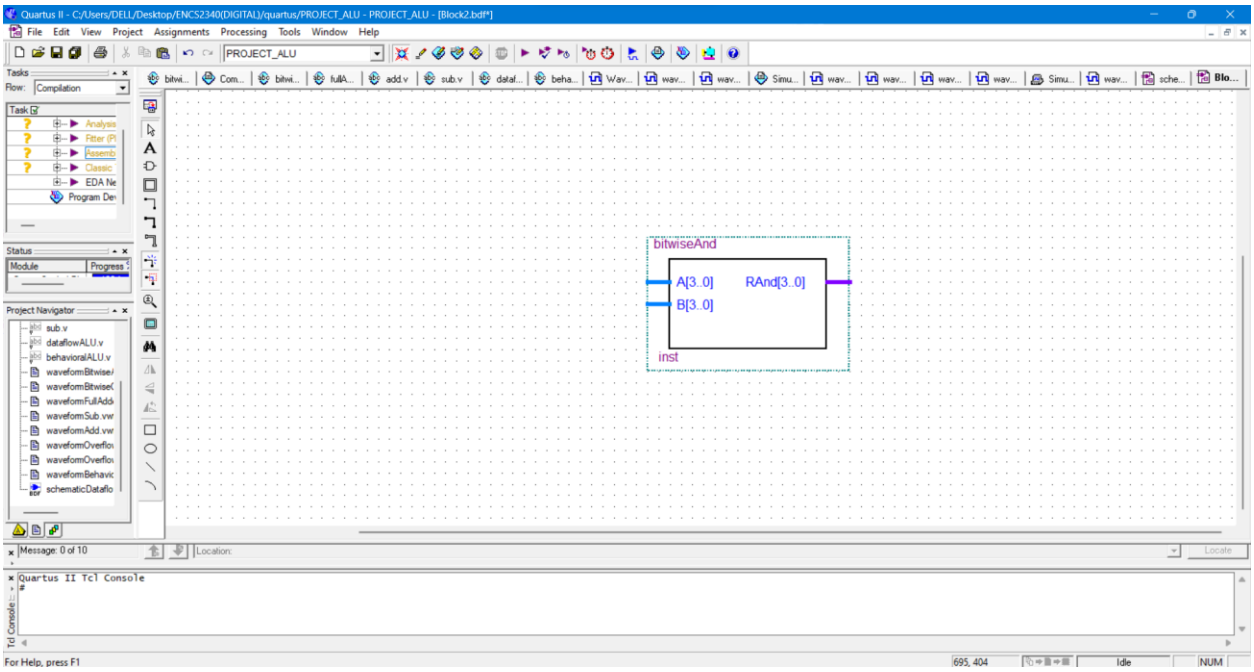


3. Simulator/ using waveform :



Here is the bitwiseAnd result from waveform as we can see that it have 2-inp And 1-out to understand the way of this module works lets take the some res, the input A=0011(because it's [3:0]A,B) and B=0110 ➔ then A[0] And B[0] =0 , A[1] And B[1] =1 , A[2] And B[2], A[3] And B[3] =0 . => so its (Rand=0010) as we can see at time (30.0 ps) the result is correct like I calculate it above.

4. Schematric for bitwiseAnd:

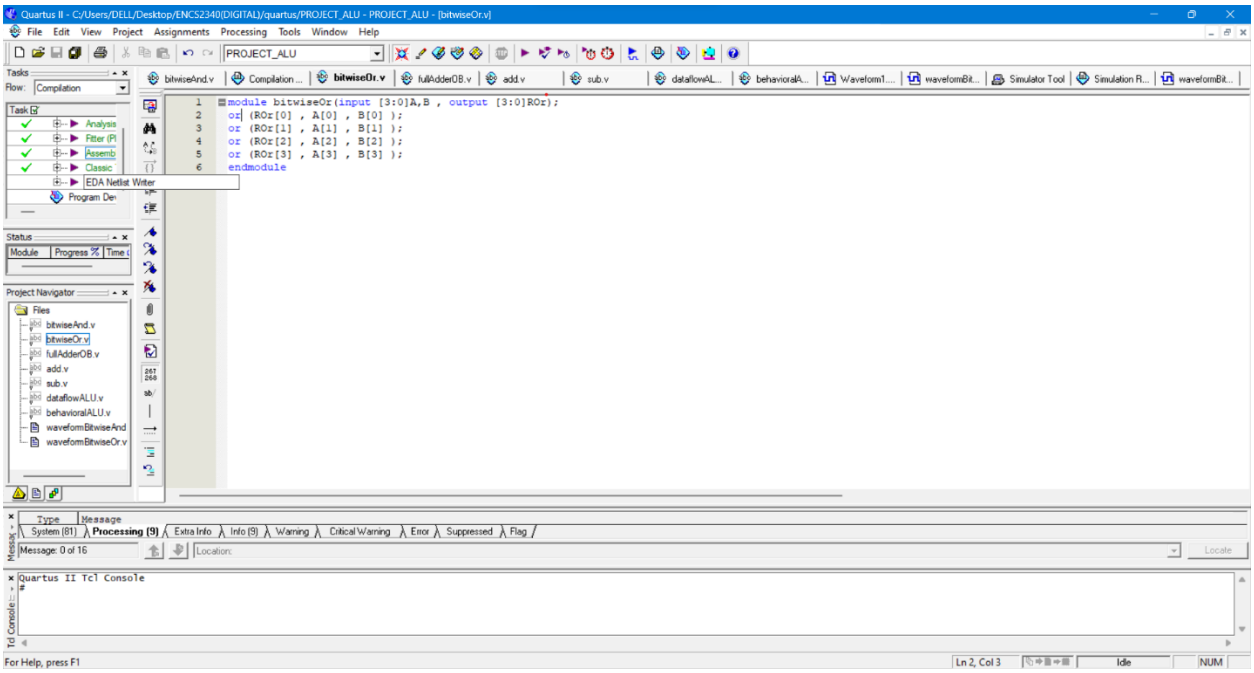


- Bitwise OR (bitwiseOr):

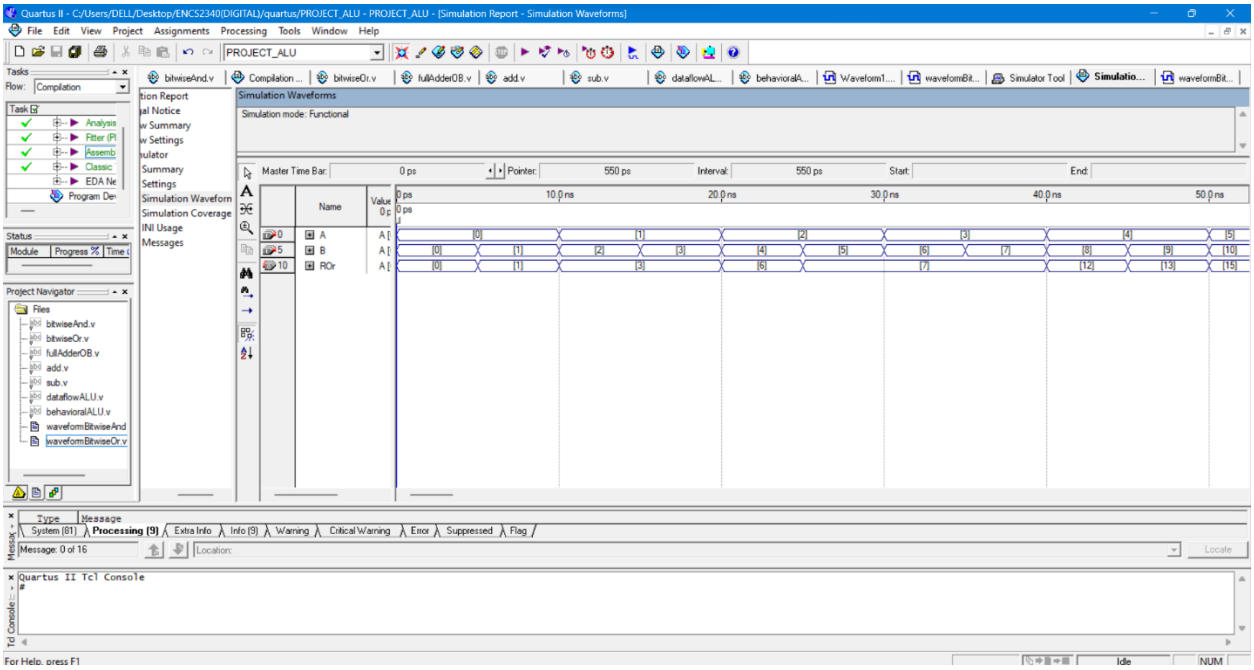
1. A **bitwise OR** is a binary operation that takes two bit patterns of equal length and performs the logical inclusive OR operation on each pair of corresponding bits. The result in each position is 0 if both bits are 0, while otherwise the result is 1

2. The code :

```
module bitwiseOr(input [3:0]A,B , output [3:0]ROr);

or (ROr[0] , A[0] , B[0] );

or (ROr[1] , A[1] , B[1] );

or (ROr[2] , A[2] , B[2] );

or (ROr[3] , A[3] , B[3] );

endmodule
```

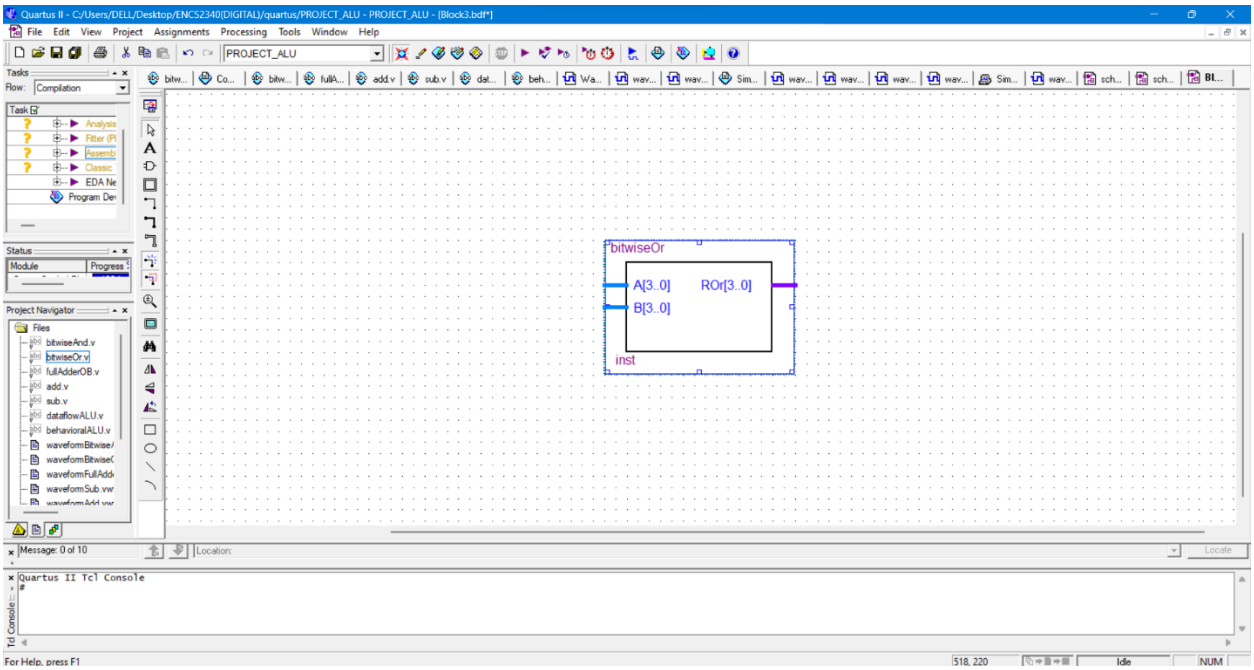


3. Simulator/ using waveform :



Here is the bitwiseOr result from waveform as we can see that it have 2-inp And 1-out to understand the way of this module works lets take the some res, the input A=0001(because it's [3:0]A,B) and B=0010 ➔ then A[0] Or B[0] =1 , A[1] Or B[1] =1 , A[2] Or B[2], A[3] Or B[3] =0 . as we can see at time (10.0 ns) the result is correct like I calculate it above.

4. Schematric for bitwiseOr:

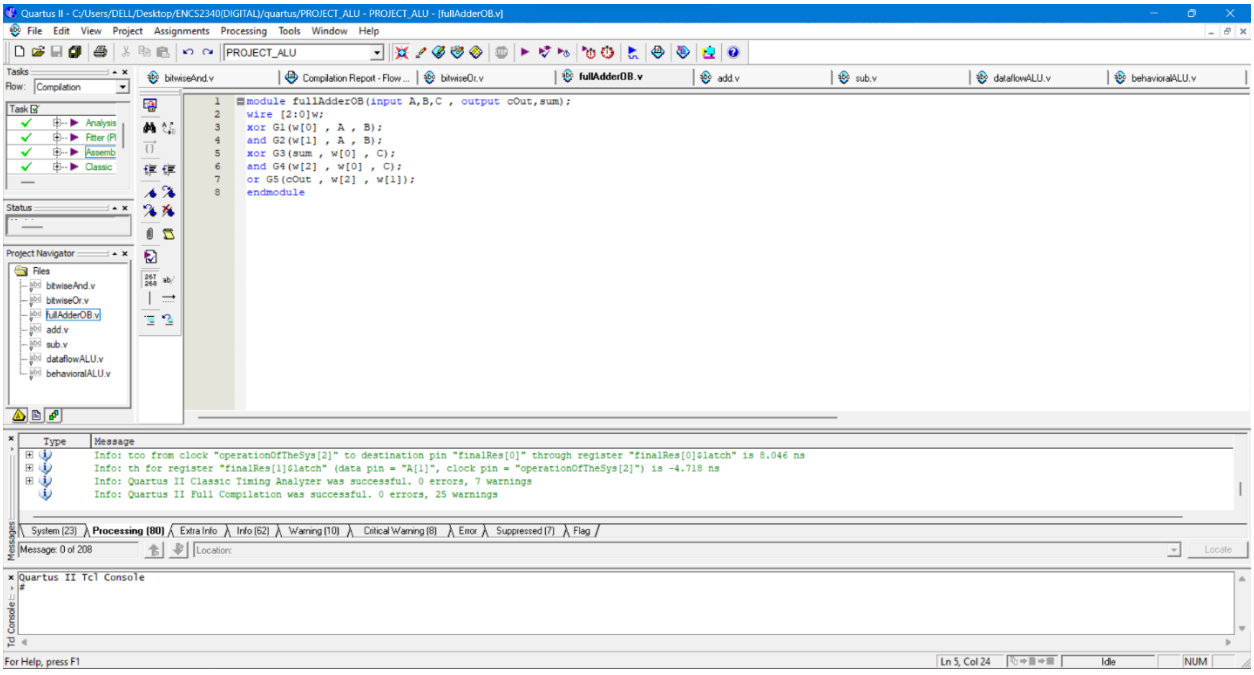


=>Links of this info for bitwiseOr/And on the starting of every one :
(https://en.wikipedia.org/wiki/Bitwise_operation)

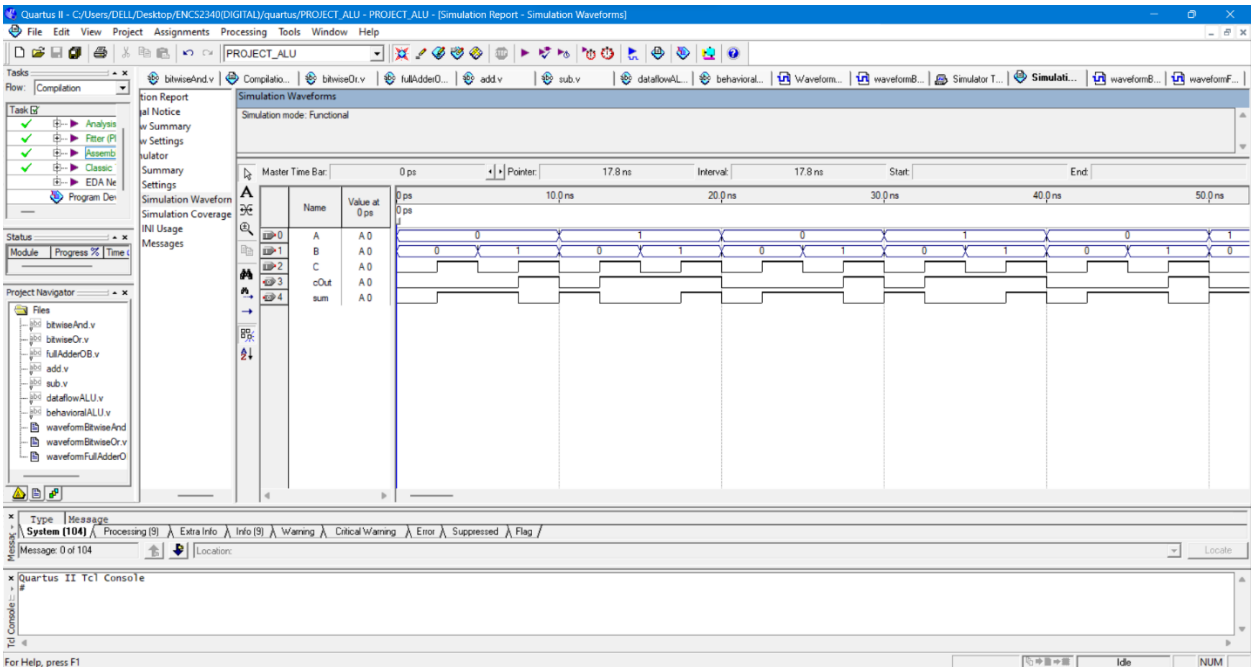
- # Full Adder (fullAdderOB):

1. Full adder adds 3 bits: x, y, and z ❖ Two output bits: 1. Carry bit: C 2. Sum bit: S ❖ Sum bit is 1 if the number of 1's in the input is odd (odd function) $\boldsymbol{S = xy'z' + x'yz' + x'y'z + xyz}$ ❖ Carry bit is 1 if the number of 1's in the input is 2 or 3 $\boldsymbol{C = xy + xz + y}$
2. ## The code :

```
module fullAdderOB(input A,B,C , output cOut,sum);
wire [2:0]w;
xor G1(w[0] , A , B);
and G2(w[1] , A , B);
xor G3(sum , w[0] , C);
and G4(w[2] , w[0] , C);
or G5(cOut , w[2] , w[1]);
endmodule
```

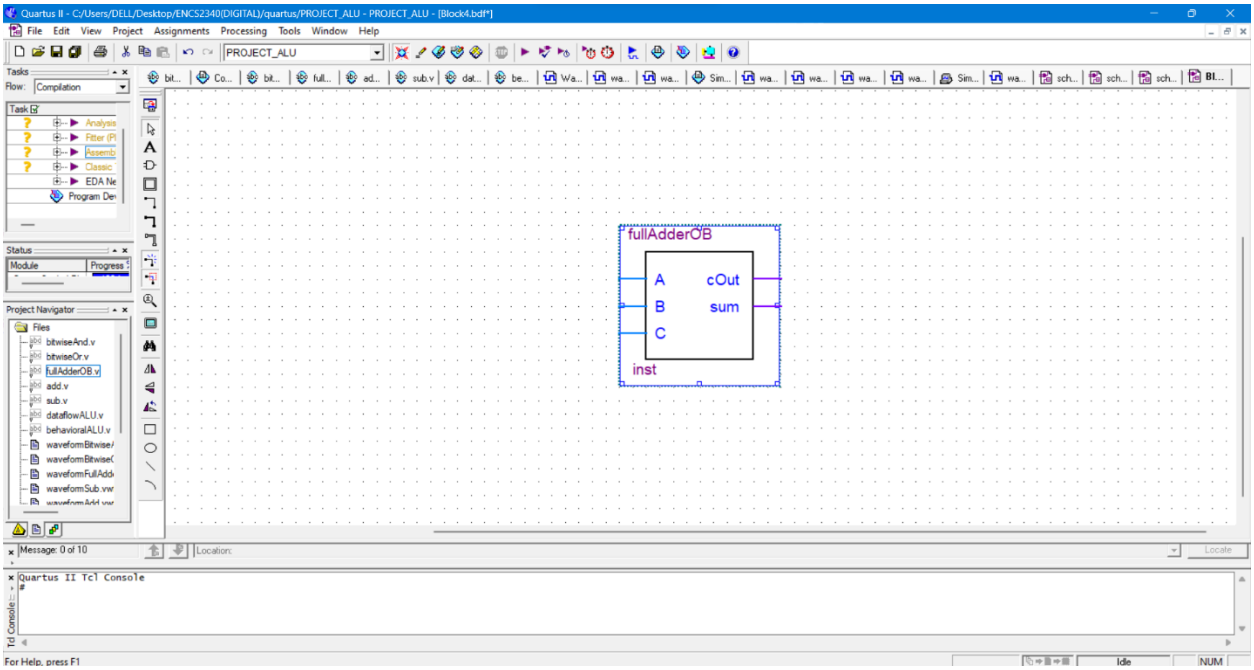


3. ## Simulator/ using waveform :



Here is the fullAdderOB result from waveform as we can see that it have 3-inp And 2-out to understand the way of this module works lets take the some res, the input A=1(because it's for 1 bit) and B=0 , C=1➔ then A+B+C => sum=0 and carry(cOut)=1 . as we can see at time (10.0 ns) the result is correct like I calculate it above.

4. Schematric for fullAdderOB:

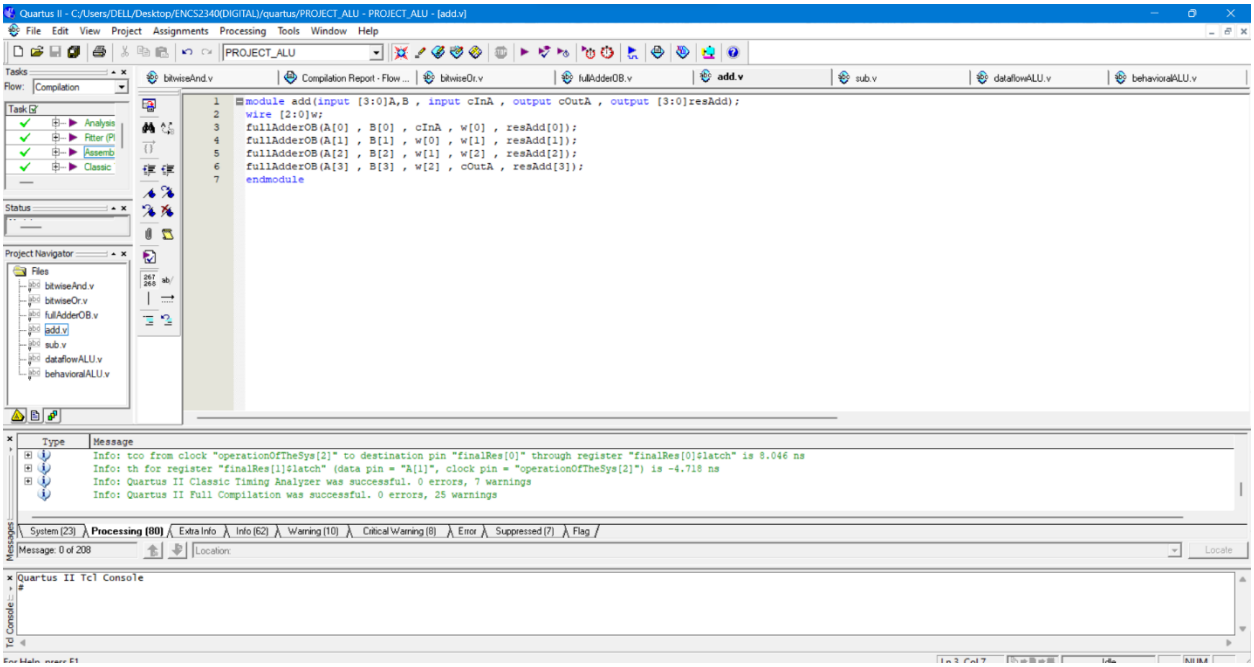


The info from Dr. Ahmed Shawahna slides

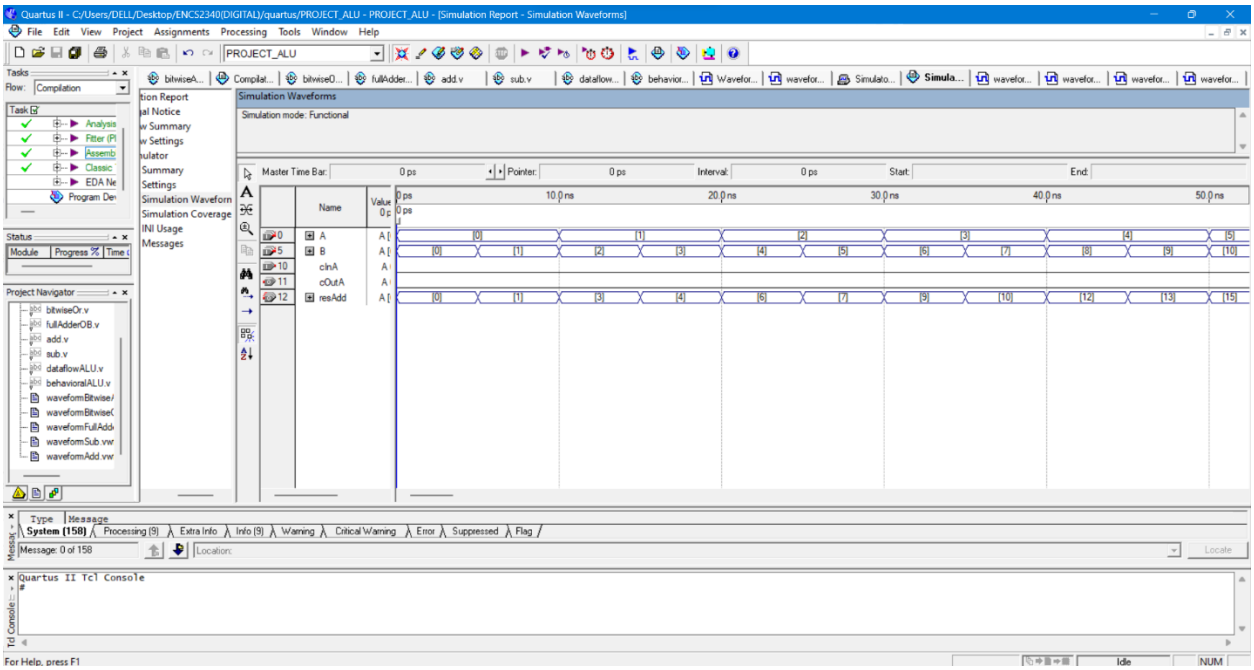
- Adder (add):

1. The module adder It is the process of adding two numbers. each number consists of 4 bits. In this design, the adder relies on using the full adder 4times module. Because we have 4-bits for each input.
2. The code :

```
module add(input [3:0]A,B , input cInA , output cOutA , output [3:0]resAdd);

wire [2:0]w;

fullAdderOB(A[0] , B[0] , cInA , w[0] , resAdd[0]);

fullAdderOB(A[1] , B[1] , w[0] , w[1] , resAdd[1]);

fullAdderOB(A[2] , B[2] , w[1] , w[2] , resAdd[2]);

fullAdderOB(A[3] , B[3] , w[2] , cOutA , resAdd[3]);

endmodule
```

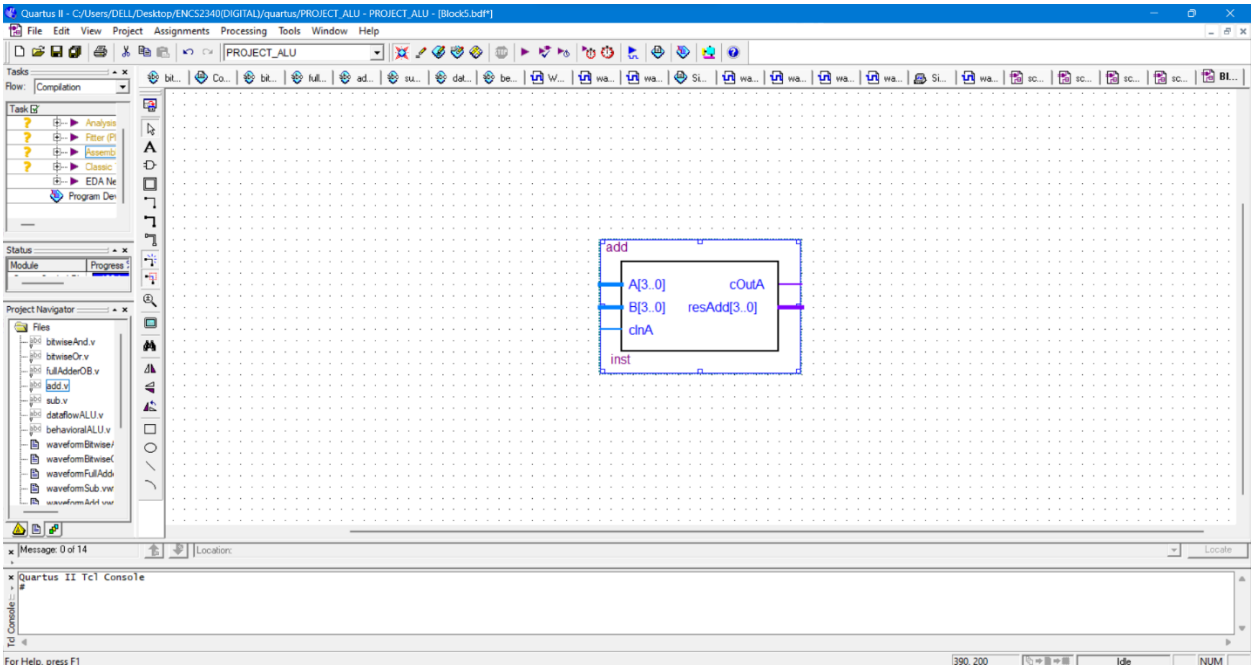


3. # Simulator/ using waveform :



Here is the adder(add) result from waveform as we can see that it have 3-inp And 2-out to understand the way of this module works lets take the some res, the input A=0001(because it's for 4- bits A and B) and B=0010 , and for the input cInA=0 always because I need it to be adder not subtractor ➔ then A+B+C => sum=0011 and carry(cOutA)=0 . as we can see at time (10.0 ns) the result is correct like I calculate it above.

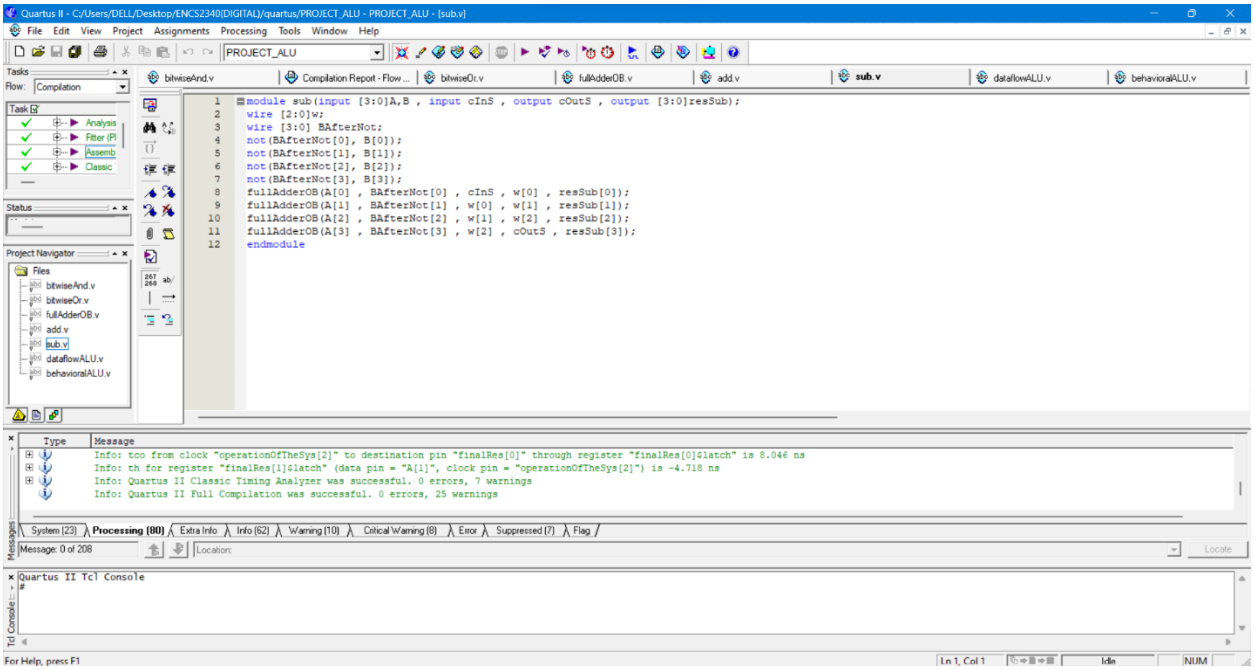
4. # Schematric for add:



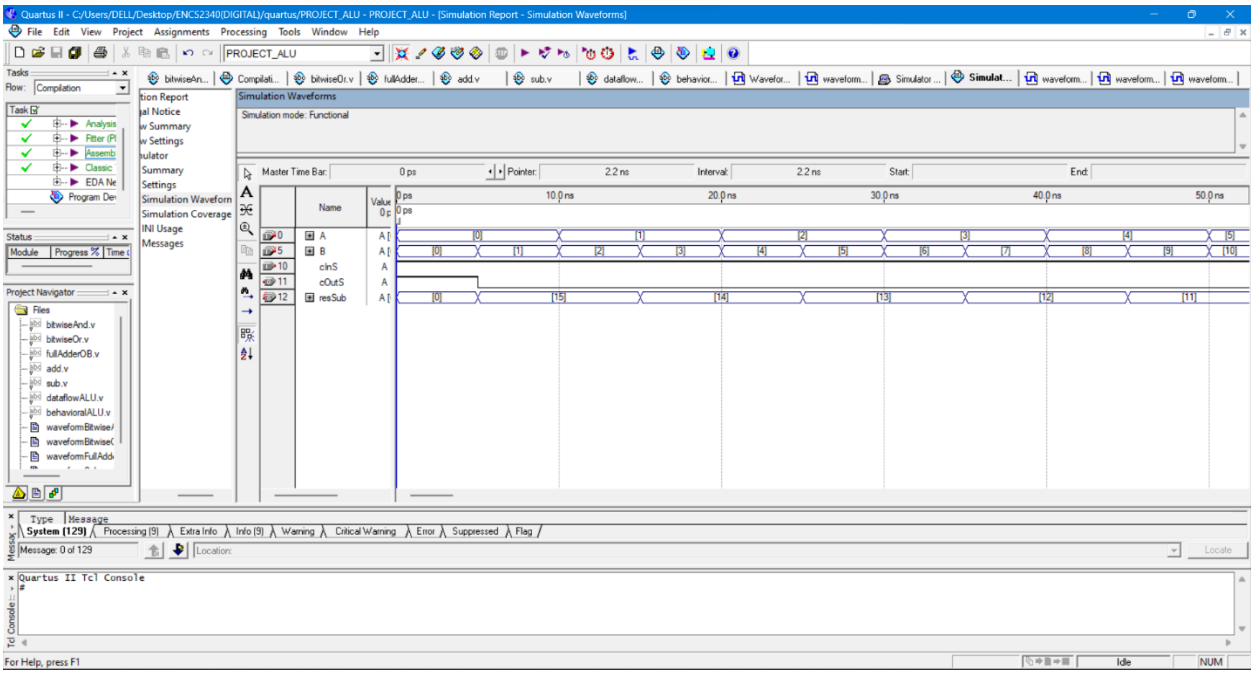
- subtractor (sub):

1. The module sub It is the process of subtract two numbers. each number consists of 4 bits. In this design, the subetractor relies on using the full adder 4times module. Because we have 4-bits for each input.
2. The code :

```
module sub(input [3:0]A,B , input cInS , output cOutS , output [3:0]resSub);

wire [2:0]w;

wire [3:0] BAfterNot; // initialize a new input that takes the 1’comp of B like down

not(BAfterNot[0], B[0]);

not(BAfterNot[1], B[1]);

not(BAfterNot[2], B[2]);

not(BAfterNot[3], B[3]); // I did that because I’m ganna put the cInS=1 to let the module subtractor

fullAdderOB(A[0] , BAfterNot[0] , cInS , w[0] , resSub[0]);

fullAdderOB(A[1] , BAfterNot[1] , w[0] , w[1] , resSub[1]);

fullAdderOB(A[2] , BAfterNot[2] , w[1] , w[2] , resSub[2]);

fullAdderOB(A[3] , BAfterNot[3] , w[2] , cOutS , resSub[3]);

endmodule
```

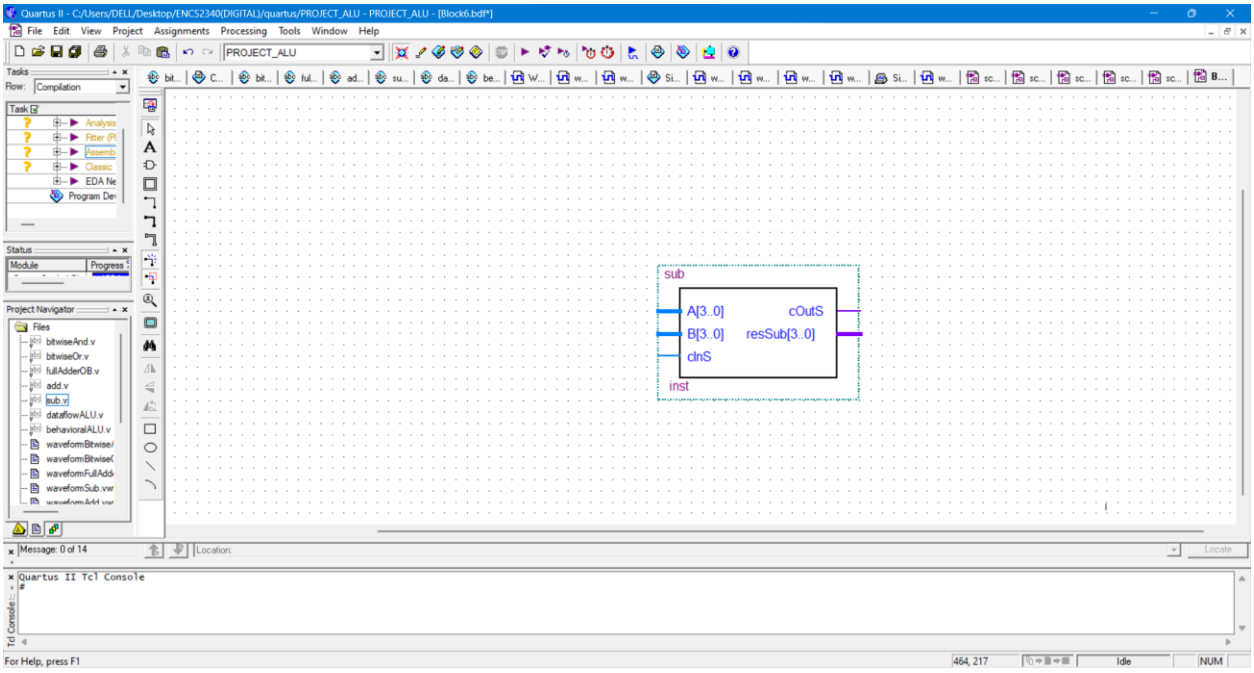


3. Simulator/ using waveform :



Here is the subtractor(sub) result from waveform as we can see that it have 3-inp And 2-out to understand the way of this module works lets take the some res, the input A=0001(because it's for 4- bits A and B) and B=0010 , and for the input cInS=1 always because I need it to be subtractor not adder => resSub=1111(15) but its not the correct so I take the 1'comp of B and let cInS is 1 always to be 2'comp(because A-B=A+2'comp of B)➔ after that if the number negative (or the most sig bit is 1 then we need to take the 2'comp of the res) and if we take the 2'comp of the ans(1111) the answer will be (-1) and that correct , and carry(cOutS)=0 . as we can see at time (10.0 ns) the result is correct like I calculate it above.

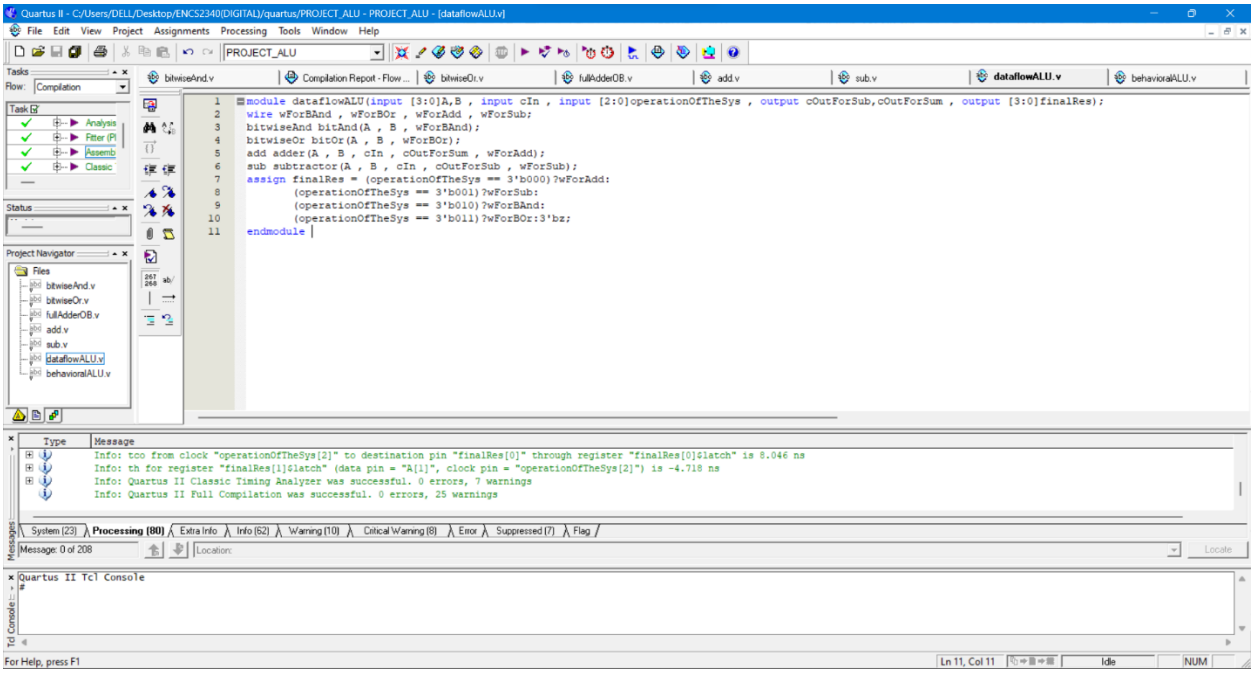
4. Schematic for sub:



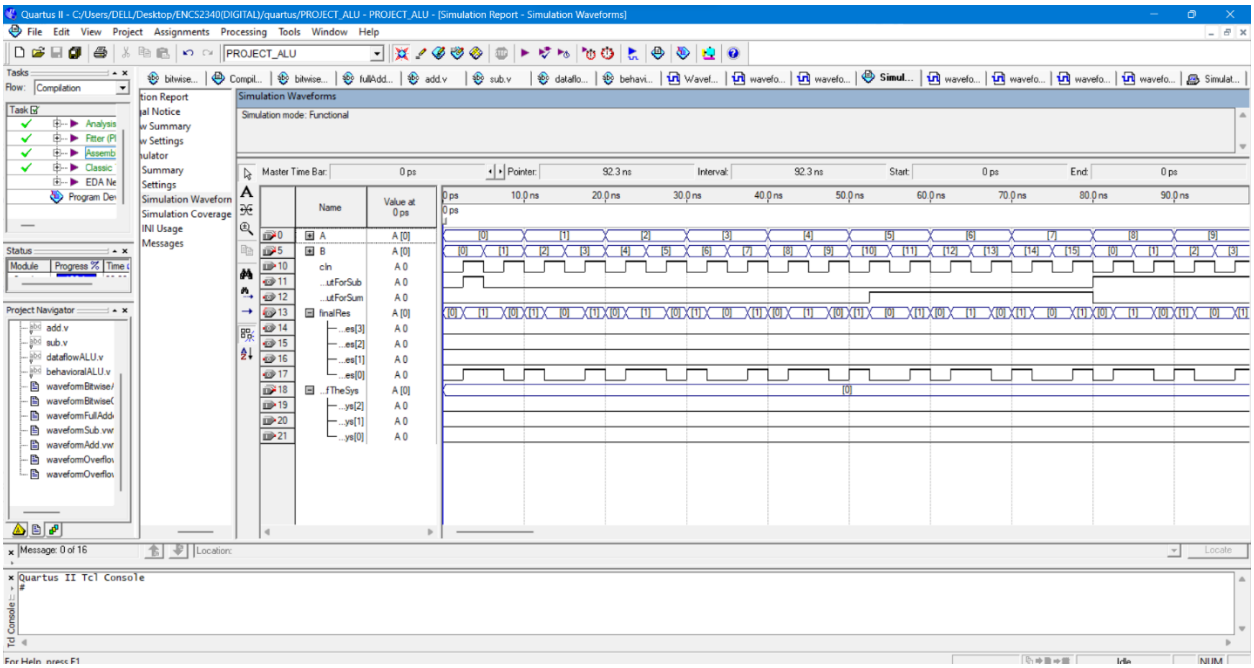
- top-level module (dataflowALU):

1. In this module, everything is collected here (or even the better word is network and installation of all modules) in this module, but in the overflow method.
2. The code :

```
module dataflowALU(input [3:0]A,B , input cIn , input [2:0]operationOfTheSys , output cOutForSub,cOutForSum , output [3:0]finalRes);

wire wForBAnd , wForBOr , wForAdd , wForSub;

bitwiseAnd bitAnd(A , B , wForBAnd); // calling the modules

bitwiseOr bitOr(A , B , wForBOr);

add adder(A , B , cIn , cOutForSum , wForAdd);

sub subtractor(A , B , cIn , cOutForSub , wForSub);

assign finalRes = (operationOfTheSys == 3'b000)?wForAdd:  // now for the opCode

    (operationOfTheSys == 3'b001)?wForSub:

    (operationOfTheSys == 3'b010)?wForBAnd:

    (operationOfTheSys == 3'b011)?wForBOr:3'bz;

endmodule
```



3. Simulator/ using waveform :



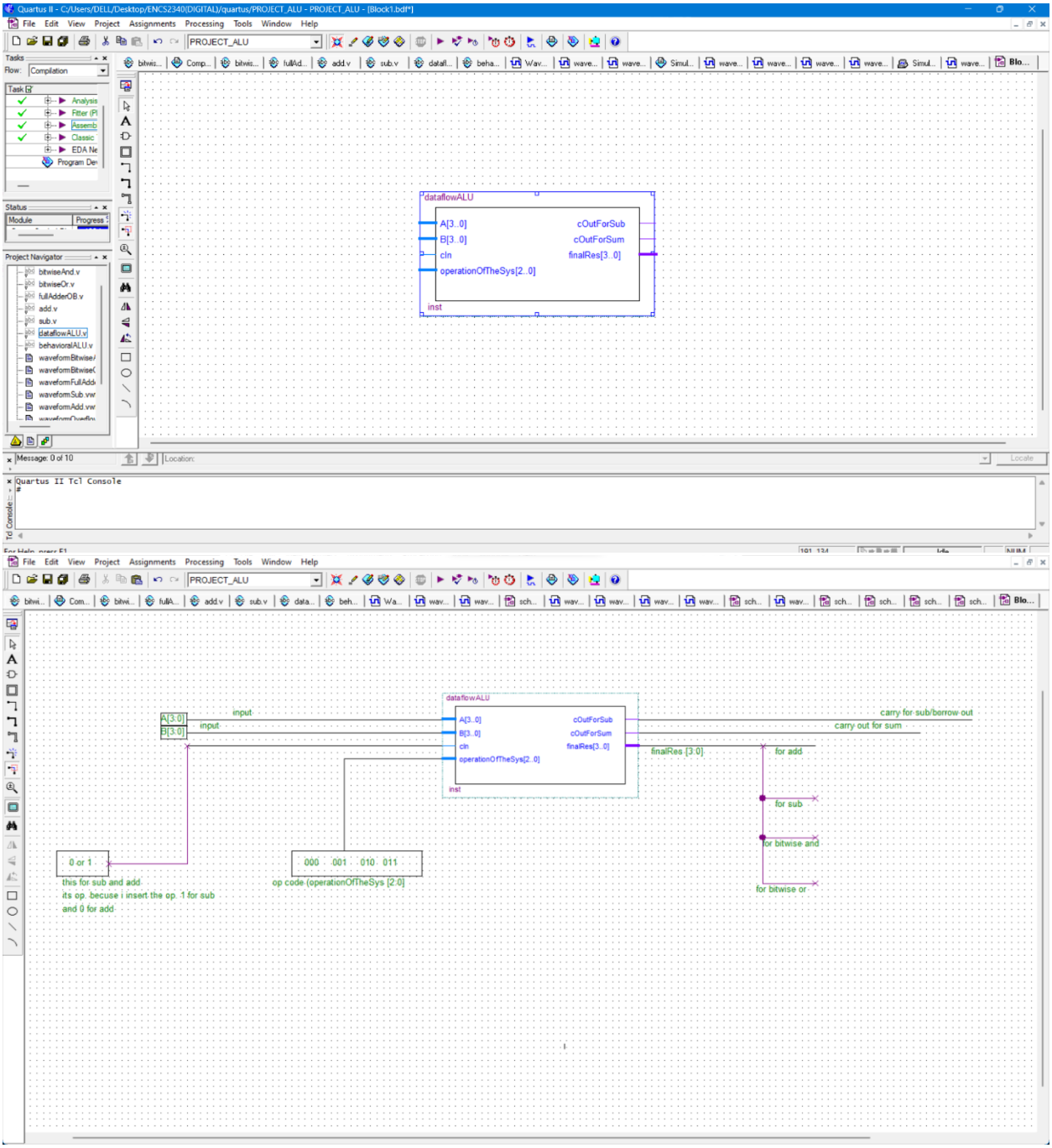
Here through it we can notice that there are 2 inputs, two caches of 4 bits each, and also input cIn, and this port is for Ader and Sub.

So that it is a single bit that determines the value of cIn entering the two modules, as mentioned previously, and also an input for choosing the appropriate operation or any operation we would like to perform on robots A and B, and it also contains two outputs, each one for each carry in sum and sub and Also an output consisting of four bits, and this is the output of the selected operation.

Let us take an example, in a = 0, b = 0, cIn 0, the selection performs the operations and gives an answer as shown, the carries in each one = 0, and when all operations are performed the answer is zero, A And B=0000 , A Or B =0000 , A+b=0000 , A-B =0000 => and the carry from both sub and add equals 0

And that correct .

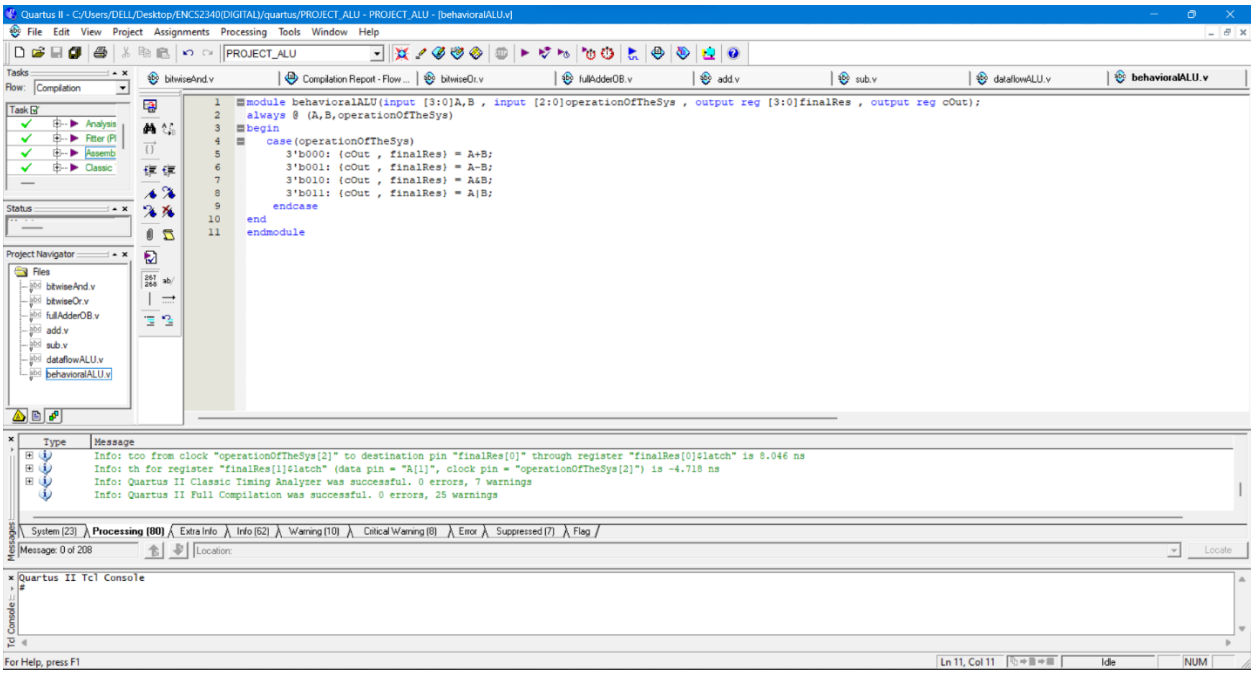
4. Schematric for overflow:



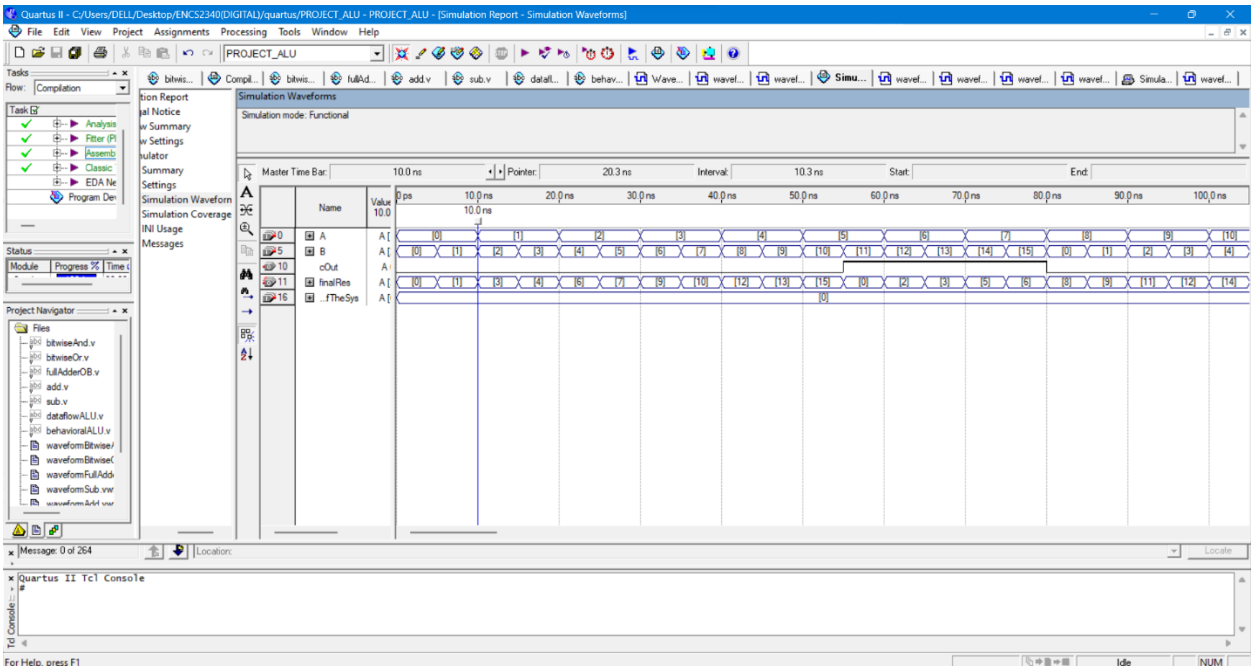
- top-level module (behavioralALU):

1. In this module, everything is collected here (or even the better word is network and installation of all modules) in this module, but in the behavioral method.
2. The code :

```
module behavioralALU(input [3:0]A,B , input [2:0]operationOfTheSys , output reg [3:0]finalRes , output reg cOut);

always @ (A,B,operationOfTheSys)

begin

  case(operationOfTheSys)

    3'b000: {cOut , finalRes} = A+B;

    3'b001: {cOut , finalRes} = A-B;

    3'b010: {cOut , finalRes} = A&B;

    3'b011: {cOut , finalRes} = A|B;

   endcase

end

endmodule
```



3. Simulator/ using waveform :



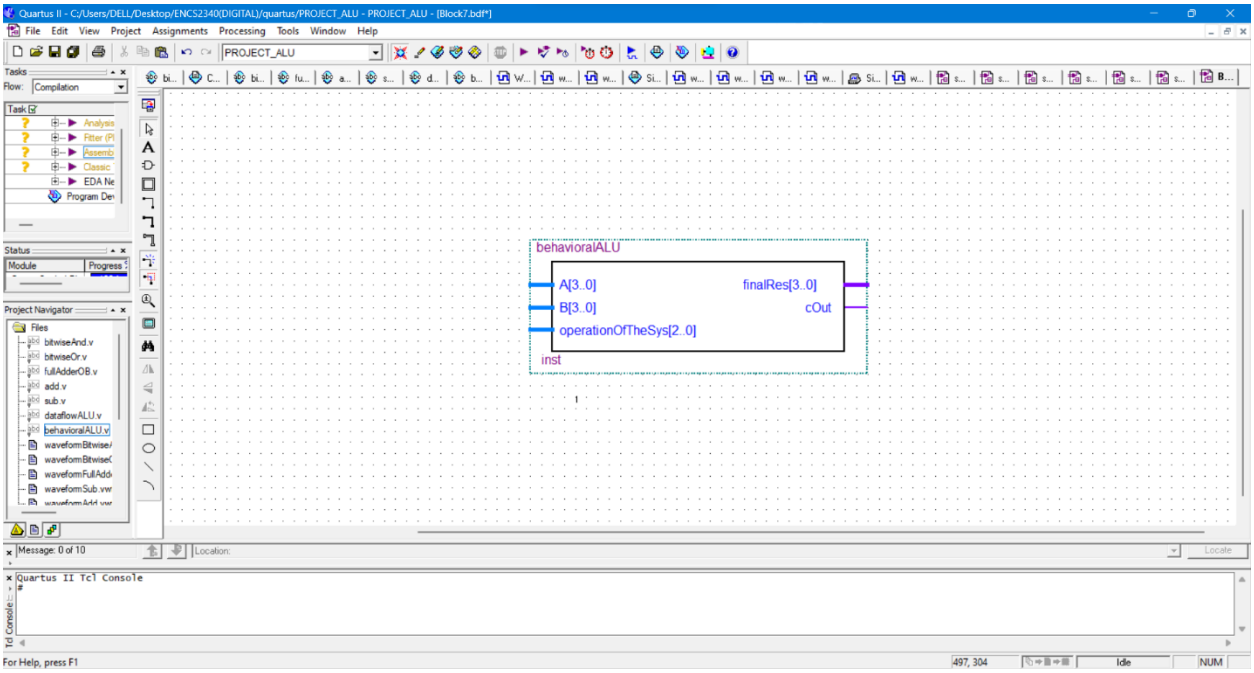
Here through it we can notice that there are 2 inputs of 4 bits each, and also input cIn, and this part is for Ader and Sub.

So that it is a single bit that determines the value of cIn entering the two modules, as mentioned previously, and also an input for choosing the appropriate operation or any operation we would like to perform on A and B, and it also contains two outputs, one for carry in sum and sub and Also an output consisting of four bits, and this is the output of the selected operation.

Let us take an example, in A = 0, b = 1, the selection performs the operations and gives an answer as shown, the carry out(cOut=0), and the final ans is 1 and theres no carry like I said before.

I want to mention that the outputs in behavioral and overflow are the same , just the way is different to do the code.

4. Schematric for behavioral:



- Conclusion and final results :

The project successfully developed a simple, functional ALU using Verilog HDL, adeptly handling basic arithmetic and logic operations. The ALU's performance, confirmed through simulations, reliably adhered to the designated OpCode, with thorough testing for scenarios like overflow ensuring robustness. The modular design approach greatly simplified debugging and testing, showcasing the effectiveness of various Verilog modeling techniques. This experience was immensely valuable, offering practical insights into digital design and hardware simulation. Future enhancements could include more complex operations and design optimizations for improved performance.

Thank you :)